Popular Frameworks for Building RAG Systems

Prepared By

SREEDEV C S

November 3, 2026

Contents

1 Introduction to RAG **1**
- 1.1 What is RAG? . . . 1
- 1.2 Need for RAG . . . 2
- 1.3 Benefits of RAG . . . 2

2 Components of a RAG Pipeline **3**
- 2.1 Document Loading . . . 4
- 2.2 Text Splitting . . . 4
- 2.3 Embedding Generation . . . 4
- 2.4 Vector Databases . . . 4
- 2.5 Retrieval . . . 5
- 2.6 Response Generation . . . 5

3 LangChain **6**
- 3.1 Introduction . . . 6
- 3.2 LangChain Architecture . . . 6
- 3.3 Document Loaders . . . 7
- 3.4 Text Splitting Strategies . . . 7
  - 3.4.1 Character Text Splitter . . . 7
  - 3.4.2 Recursive Character Splitter . . . 8
  - 3.4.3 Token-Based Splitter . . . 8
- 3.5 Embedding Models . . . 8
- 3.6 Retrievers . . . 8
  - 3.6.1 Similarity Search . . . 8
  - 3.6.2 Maximum Marginal Relevance (MMR) . . . 9
  - 3.6.3 Parent-Child Retrieval . . . 9
  - 3.6.4 Multi-Query Retrieval . . . 9
  - 3.6.5 Contextual Compression . . . 9
- 3.7 LangChain Expression Language (LCEL) . . . 9
- 3.8 Agents and Tool Calling . . . 9
- 3.9 RAG Workflow . . . 10
- 3.10 Real-World Use Cases . . . 10

4 LlamaIndex **11**
4.1 Introduction . . . 11
4.2 Data Model . . . 11
4.3 Node Parsing . . . 11
4.4 Index Structures . . . 12
4.4.1 Vector Store Index . . . 12
4.4.2 Tree Index . . . 12
4.4.3 Keyword Table Index . . . 12
4.4.4 Knowledge Graph Index . . . 12
4.5 Query Engine . . . 12
4.6 Advanced Retrieval Techniques . . . 13
4.7 Document Processing . . . 13
4.8 Real-World Use Cases . . . 13
4.9 Advantages . . . 14

5 Haystack **15**
5.1 Introduction . . . 15
5.2 Pipeline Architecture . . . 15
5.3 Document Stores . . . 15
5.4 Retrieval Strategies . . . 16
5.4.1 Sparse Retrieval . . . 16
5.4.2 Dense Retrieval . . . 16
5.4.3 Hybrid Retrieval . . . 16
5.5 Ranking and Re-Ranking . . . 16
5.6 Production Deployment . . . 17
5.7 Real-World Use Cases . . . 17
5.8 Advantages . . . 17

6 Practical RAG Applications **18**
6.1 Document Question Answering . . . 18
6.2 Customer Support Chatbots . . . 18
6.3 Enterprise Knowledge Management . . . 19
6.4 Legal Document Analysis . . . 19
6.5 Healthcare Information Systems . . . 20
6.6 Code Assistant Systems . . . 20
6.7 Educational and Learning Platforms . . . 20

7 Comparison of RAG Frameworks **21**

Chapter 1

Introduction to RAG

1.1 What is RAG?

Retrieval-Augmented Generation (RAG) is an architectural pattern that combines the knowledge retrieval capabilities of information retrieval systems with the reasoning and language generation capabilities of Large Language Models (LLMs). Instead of relying solely on the knowledge stored in model parameters during training, a RAG system retrieves relevant information from external knowledge sources and incorporates it into the prompt before generating a response [2].

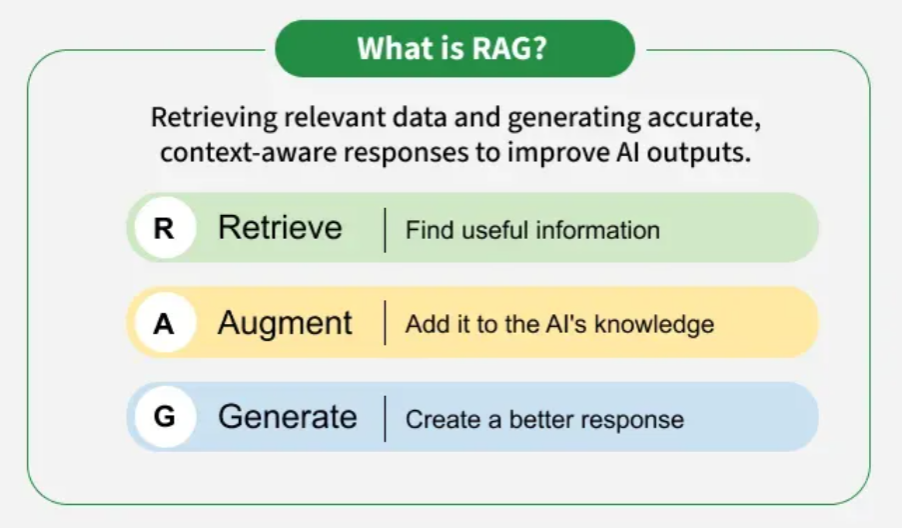


Figure 1.1: Retrieval Augmented Generation

The popularity of RAG has grown rapidly because organizations often possess proprietary documents, manuals, reports, and databases that are not included in the training data of general-purpose Large Language Models. RAG enables language models to access and utilize such information dynamically.

1.2 Need for RAG

The major problems RAG solve are [2]:

- Hallucinations: Traditional generative models can produce incorrect information. RAG reduces this risk by retrieving verified, external data to ground responses in factual knowledge.
- Outdated Information: Static models rely on training data that may become outdated. RAG dynamically retrieves latest information ensuring relevance and accuracy in real time.
- Contextual Relevance: Generative models often struggle with maintaining context in complex or multi turn conversations. RAG retrieves relevant documents to enrich the context improving coherence and relevance.
- Domain Specific Knowledge: Generic models may lack expertise in specialized fields. RAG integrates domain specific external knowledge for tailored and precise responses.
- Cost and Efficiency: Fine tuning large models for specific tasks is expensive. RAG eliminates the need for retraining by dynamically retrieving relevant data reducing costs and computational load.
- Scalability Across Domains: RAG is adaptable to diverse industries from healthcare to finance without extensive retraining making it highly scalable.

1.3 Benefits of RAG

The major benefits of RAG include:

- Improved factual accuracy.
- Access to real-time information.
- Usage of organization-specific knowledge.
- Reduced hallucinations.
- Lower maintenance costs compared to model retraining.

These advantages make RAG one of the most widely adopted approaches for enterprise AI applications [3].

Chapter 2

Components of a RAG Pipeline

A Retrieval-Augmented Generation system consists of multiple interconnected components that transform raw documents into searchable knowledge and utilize them during answer generation.

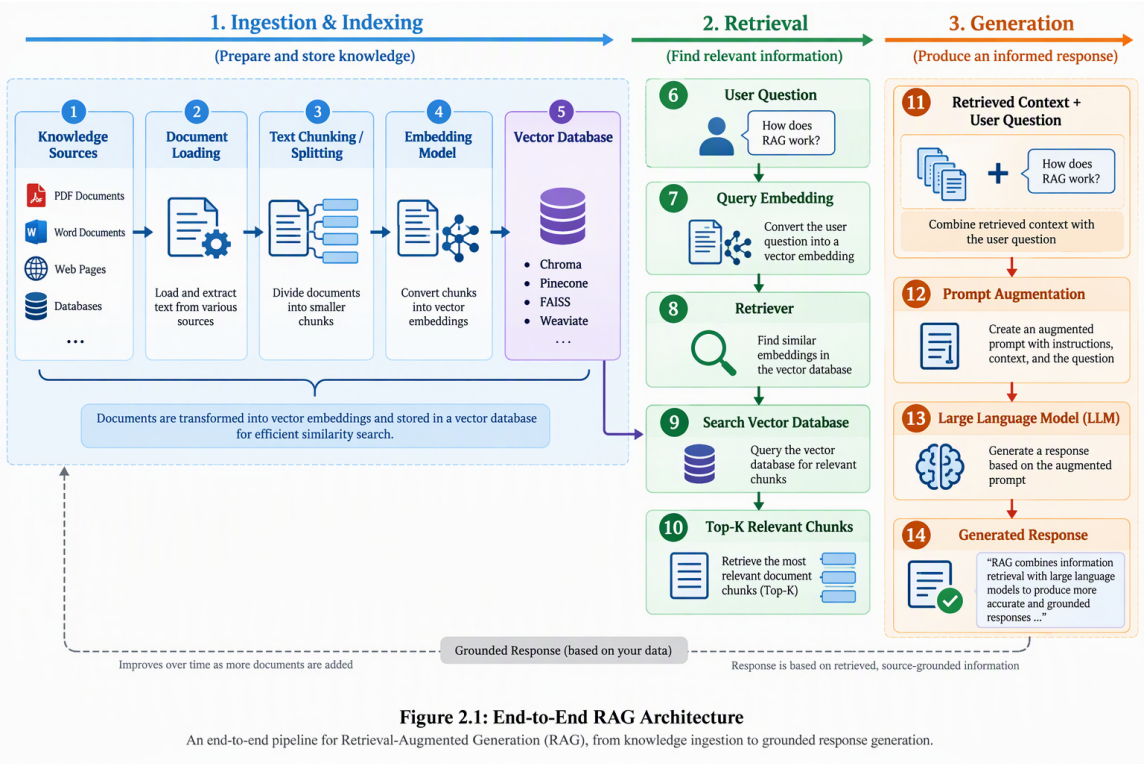


Figure 2.1: End-to-End RAG Architecture
An end-to-end pipeline for Retrieval-Augmented Generation (RAG), from knowledge ingestion to grounded response generation.

Figure 2.1: End-to-End RAG Architecture

2.1 Document Loading

Document loading is the process of importing data from various sources such as PDF files, Word documents, web pages, databases, and APIs. Frameworks typically provide specialized loaders capable of extracting textual content from these sources.

2.2 Text Splitting

Large documents are divided into smaller chunks before indexing. Chunking ensures that retrieved passages remain relevant and fit within the context window of language models. Common chunking strategies include:

- Fixed-size chunking
- Recursive chunking
- Semantic chunking
- Sentence-based chunking

2.3 Embedding Generation

Embeddings are numerical vector representations of text. Embedding models transform textual information into dense vectors where semantically similar content appears close together in vector space.

2.4 Vector Databases

Vector databases store embeddings and support efficient similarity search. Popular examples include:

- Pinecone
- Chroma
- Weaviate
- Milvus
- FAISS

2.5 Retrieval

When a user submits a query, the system converts the query into an embedding and searches the vector database to retrieve the most relevant document chunks.

2.6 Response Generation

Retrieved content is combined with the user query and supplied to a Large Language Model. The model generates a response grounded in the retrieved context, improving factual accuracy and reducing hallucinations.

Chapter 3

LangChain

3.1 Introduction

LangChain is an open-source framework designed for building applications powered by Large Language Models (LLMs). It provides abstractions that simplify the integration of prompts, language models, retrieval systems, external tools, memory components, and agents into a unified workflow [5].

Unlike frameworks that focus primarily on retrieval, LangChain acts as an orchestration layer between multiple AI components. It enables developers to construct complex applications such as Retrieval-Augmented Generation systems, conversational assistants, autonomous agents, and workflow automation tools.

3.2 LangChain Architecture

The architecture of LangChain is based on modular components that can be composed together. The major layers include:

1. Data Ingestion Layer
2. Processing Layer
3. Retrieval Layer
4. LLM Layer
5. Agent and Tool Layer
6. Application Layer

3.3 Document Loaders

Document loaders are responsible for acquiring data from external sources. LangChain supports a wide variety of loaders, including:

- PDFLoader
- TextLoader
- CSVLoader
- WebBaseLoader
- ConfluenceLoader
- SharePoint Loader
- DirectoryLoader

Each loader converts raw data into Document objects. A Document object generally contains:

- Page Content
- Metadata

Metadata may include source information, author names, timestamps, and document identifiers. This metadata can later be used during retrieval filtering.

3.4 Text Splitting Strategies

Large Language Models have context window limitations. Therefore documents must be divided into smaller chunks before indexing. LangChain provides several splitting approaches:

3.4.1 Character Text Splitter

Divides documents using a fixed number of characters.

3.4.2 Recursive Character Splitter

The most commonly used approach. It recursively splits text using:

- Paragraph boundaries
- Sentence boundaries
- Word boundaries

while preserving semantic structure.

3.4.3 Token-Based Splitter

Splits documents according to model tokenizer boundaries, ensuring compatibility with model context limits.

3.5 Embedding Models

Once documents are chunked, each chunk is converted into a dense vector representation known as an embedding. Common embedding providers include:

- OpenAI Embeddings
- HuggingFace Embeddings
- Cohere Embeddings
- Sentence Transformers

Embeddings capture semantic meaning rather than exact keywords. Similar textual concepts therefore appear closer together in vector space.

3.6 Retrievers

Retrievers are responsible for finding relevant chunks from the knowledge base. Popular retrieval strategies include:

3.6.1 Similarity Search

Retrieves chunks whose embeddings have the highest cosine similarity to the query embedding.

3.6.2 Maximum Marginal Relevance (MMR)

Balances relevance and diversity by reducing duplicate retrievals.

3.6.3 Parent-Child Retrieval

Stores small chunks while retrieving larger parent documents to preserve context.

3.6.4 Multi-Query Retrieval

Generates multiple query reformulations using an LLM to improve recall.

3.6.5 Contextual Compression

Removes irrelevant information from retrieved documents before sending them to the language model.

3.7 LangChain Expression Language (LCEL)

Modern LangChain applications are developed using the LangChain Expression Language (LCEL). LCEL enables developers to compose complex AI workflows using operator-based chaining. An example workflow consists of:

$$\text{Prompt} \rightarrow \text{Retriever} \rightarrow \text{Context Formatter} \rightarrow \text{LLM} \rightarrow \text{Output Parser}$$

LCEL standardizes component interaction and supports asynchronous execution, streaming, batching, and observability.

3.8 Agents and Tool Calling

LangChain agents allow language models to dynamically decide which actions should be executed. An agent can select among multiple tools such as:

- Web Search
- Calculators
- Database Queries

- API Calls
- Vector Search

The reasoning loop generally follows:

$$\text{Thought} \rightarrow \text{Action} \rightarrow \text{Observation} \rightarrow \text{Thought} \rightarrow \text{Final Answer}$$

3.9 RAG Workflow

A typical LangChain-based RAG workflow consists of the following stages:

1. Load documents.
2. Split documents into chunks.
3. Generate embeddings.
4. Store embeddings in a vector database.
5. Retrieve relevant chunks.
6. Pass retrieved context to an LLM.
7. Generate the final response.

LangChain acts as the orchestration layer connecting all these components [4].

3.10 Real-World Use Cases

LangChain is particularly suitable when multiple AI components need to be orchestrated together. The framework supports chaining, tool integration, memory management, and agent-based workflows. Common use cases include:

- Enterprise document chatbots
- Customer support assistants
- Multi-step AI workflows
- AI agents that access external tools
- Research assistants
- Knowledge-base question answering systems

Chapter 4

LlamaIndex

4.1 Introduction

LlamaIndex is a data framework specialized for connecting enterprise data sources with Large Language Models. The framework focuses on indexing, retrieval, query execution, and data organization for Retrieval-Augmented Generation systems [6]. Whereas LangChain primarily orchestrates workflows, LlamaIndex primarily optimizes how information is represented and retrieved.

4.2 Data Model

LlamaIndex organizes information using three hierarchical abstractions:

1. Documents
2. Nodes
3. Indexes

Documents represent source data. Nodes are smaller semantic units extracted from documents. Indexes organize nodes to enable efficient retrieval.

4.3 Node Parsing

Node parsing is a fundamental concept in LlamaIndex. Unlike simple chunking approaches, node parsers preserve additional metadata including:

- Source relationships

- Parent-child relationships
- Section hierarchy
- Reference links

This richer representation enables advanced retrieval strategies compared with traditional chunk-based systems.

4.4 Index Structures

LlamaIndex supports multiple index implementations.

4.4.1 Vector Store Index

Most commonly used index for semantic retrieval.

4.4.2 Tree Index

Organizes information hierarchically, enabling recursive retrieval and summarization.

4.4.3 Keyword Table Index

Supports keyword-based retrieval.

4.4.4 Knowledge Graph Index

Constructs graph relationships among entities and concepts.

4.5 Query Engine

A Query Engine transforms user requests into retrieval operations. The execution pipeline consists of:

1. Query Understanding
2. Retrieval
3. Post Processing
4. Response Synthesis

4.6 Advanced Retrieval Techniques

LlamaIndex supports sophisticated retrieval mechanisms:

- Recursive Retrieval
- Hybrid Search
- Metadata Filtering
- Auto-Retrieval
- Query Transformation
- Graph Retrieval

These techniques significantly improve retrieval quality in enterprise-scale deployments.

4.7 Document Processing

Documents are transformed into nodes, which are smaller units of information. These nodes are organized into various index structures that enable efficient retrieval.

4.8 Real-World Use Cases

LlamaIndex specializes in indexing, retrieval, and querying large collections of documents. It is often selected when efficient retrieval is more important than workflow orchestration. Typical applications include:

- Research paper search systems
- Financial report analysis
- Internal company knowledge assistants
- Large-scale documentation search
- Semantic document retrieval
- Data exploration systems

4.9 Advantages

- Optimized specifically for RAG.
- Multiple indexing strategies.
- Flexible retrieval techniques.
- Easy integration with LLM providers.

Chapter 5

Haystack

5.1 Introduction

Haystack is an open-source framework developed by deepset for creating production-grade search, retrieval, and question-answering systems [1]. The framework emphasizes pipeline construction, scalability, deployment flexibility, and retrieval optimization.

5.2 Pipeline Architecture

The central abstraction in Haystack is the Pipeline. A pipeline is represented as a directed graph in which nodes exchange information. Typical nodes include:

- Document Stores
- Retrievers
- Readers
- Rankers
- Prompt Builders
- Generators

5.3 Document Stores

Document Stores hold indexed knowledge and provide efficient retrieval. Supported options include:

- Elasticsearch
- OpenSearch
- PostgreSQL
- Pinecone
- Weaviate
- FAISS

Document Stores act as the persistent layer of the RAG architecture.

5.4 Retrieval Strategies

Haystack supports multiple retrieval approaches.

5.4.1 Sparse Retrieval

Keyword-based retrieval methods such as BM25.

5.4.2 Dense Retrieval

Embedding-based semantic retrieval.

5.4.3 Hybrid Retrieval

Combines sparse and dense retrieval for higher accuracy. Hybrid retrieval is commonly used in enterprise applications because it balances exact keyword matching with semantic understanding.

5.5 Ranking and Re-Ranking

After retrieval, ranking models reorder results. Common ranking approaches include:

- Cross Encoder Ranking
- Transformer-Based Ranking
- Metadata-Based Ranking

Re-ranking improves answer quality by prioritizing the most relevant passages.

5.6 Production Deployment

Haystack is designed for large-scale deployments. Key production capabilities include:

- REST API Integration
- Containerized Deployment
- Kubernetes Support
- Distributed Indexing
- Monitoring and Logging

These capabilities make Haystack suitable for enterprise-grade search and Retrieval-Augmented Generation systems.

5.7 Real-World Use Cases

Haystack is widely used in production-grade search and question-answering systems. Its modular pipeline architecture makes it suitable for enterprise deployments. Typical use cases include:

- Production search engines
- FAQ assistants
- Enterprise question answering
- Legal document retrieval
- Healthcare information retrieval
- Hybrid search systems combining keyword and semantic search

5.8 Advantages

- Production readiness.
- Strong search capabilities.
- Flexible component pipelines.
- Support for hybrid search.

Chapter 6

Practical RAG Applications

Retrieval-Augmented Generation has become a foundational architecture for enterprise AI applications. Organizations utilize RAG systems to provide accurate, context-aware, and up-to-date responses by grounding Large Language Models with external knowledge sources.

6.1 Document Question Answering

One of the most common applications of RAG is document question answering. Organizations often possess large collections of PDF documents, manuals, contracts, and reports. A RAG system can retrieve relevant sections from these documents and generate precise answers to user queries. Examples include:

- Employee policy assistants
- Technical documentation chatbots
- Research paper assistants
- Product manuals and support systems

LangChain and LlamaIndex are frequently used for such applications because of their document processing capabilities.

6.2 Customer Support Chatbots

Traditional chatbots rely on predefined responses and often struggle with complex customer queries. RAG-based chatbots can retrieve information

from knowledge bases, FAQs, documentation portals, and support tickets before generating a response. Benefits include:

- Reduced hallucinations
- More accurate answers
- Real-time knowledge updates
- Lower support costs

6.3 Enterprise Knowledge Management

Large enterprises store information across SharePoint sites, internal wikis, databases, and document repositories. RAG systems can unify access to these sources and provide employees with a natural language interface for information retrieval. Typical examples include:

- HR knowledge assistants
- IT support assistants
- Project documentation search systems
- Compliance information retrieval

6.4 Legal Document Analysis

Law firms and legal departments manage thousands of contracts and legal documents. RAG systems can retrieve relevant clauses, regulations, and precedents before generating summaries or answering questions. Common tasks include:

- Contract analysis
- Compliance verification
- Case law retrieval
- Legal research assistance

6.5 Healthcare Information Systems

Healthcare organizations can employ RAG systems to assist clinicians by retrieving information from medical literature, treatment guidelines, and patient records. Potential applications include:

- Clinical decision support
- Medical literature search
- Treatment recommendation assistance
- Healthcare knowledge retrieval

6.6 Code Assistant Systems

RAG can improve software development assistants by retrieving source code, API documentation, design specifications, and coding standards before generating recommendations. Applications include:

- Internal coding assistants
- API documentation chatbots
- Codebase search systems
- Specification-driven development assistants

6.7 Educational and Learning Platforms

Educational institutions and online learning platforms can create intelligent tutors that retrieve course material, lecture notes, and textbooks before generating explanations. Examples include:

- Personalized learning assistants
- Interactive course tutors
- University knowledge portals
- Training content assistants

Chapter 7

Comparison of RAG Frameworks

Feature	**LangChain**	**LlamaIndex**	**Haystack**
Primary Goal	Workflow Orchestration	Data Indexing	Production Pipelines
Core Abstraction	Chains / Agents	Indexes / Nodes	Pipelines
Best For	Agent Systems	Document Retrieval	Enterprise Search
Advanced Retrieval	Good	Excellent	Excellent
Knowledge Graph Support	Limited	Strong	Moderate
Agent Support	Excellent	Moderate	Limited
Production Readiness	High	High	Very High

Table 7.1: Technical Comparison of Major RAG Frameworks

Bibliography

[1] deepset. *Haystack Documentation*. Accessed: 2026-11-03. 2025. URL: https://docs.haystack.deepset.ai/.

[2] GeeksforGeeks. *Retrieval Augmented Generation (RAG)*. URL: https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/.

[3] IBM. *Retrieval-Augmented Generation (RAG)*. Accessed: 2026-11-03. 2025. URL: https://www.ibm.com/topics/retrieval-augmented-generation.

[4] LangChain. *Build a RAG Application with LangChain*. Accessed: 2026-11-03. 2025. URL: https://python.langchain.com/docs/tutorials/rag/.

[5] LangChain. *LangChain Documentation*. Accessed: 2026-11-03. 2025. URL: https://python.langchain.com/docs/.

[6] LlamaIndex. *LlamaIndex Documentation*. Accessed: 2026-11-03. 2025. URL: https://docs.llamaindex.ai/.